CHAPTER 6

Frontend Framework

Vue.js

3-Hour Lecture and Lab Session

Learning Outcomes

By the end of this chapter, students will be able to...

1. **Explain the role of frontend frameworks**

   Understand the problems solved by frameworks vs. plain JavaScript/jQuery.

2. **Describe the Vue.js ecosystem**

   Identify Vue's key features, philosophy, and where it fits among modern frameworks.

3. **Build a Vue instance**

   Use createApp() and reactive state to make dynamic web pages.

4. **Apply core directives**

   Use v-bind, v-on, v-model, v-if, v-for to make interactive UIs.

Why Frontend Frameworks?

Plain JavaScript struggles as apps grow

Plain JavaScript / jQuery

- Manual DOM updates for every state change
- UI and data easily get out of sync
- Hundreds of lines of spaghetti code
- Hard to reuse UI pieces (no components)
- Difficult to test and maintain

Modern Framework (Vue, React, Angular)

- Declarative UI ("what" not "how")
- Reactive — data drives the DOM
- Component-based — reusable, composable
- Built-in routing, state, build tools
- Large ecosystem and community

Frameworks let you describe the UI as a function of data — the framework handles the DOM.

Declarative vs. Imperative

Same task — toggle a counter — two very different code styles

Imperative (Vanilla JS)

```
let count = 0;
const btn = document.getElementById("b");
const out = document.getElementById("o");

btn.addEventListener("click", () => {
  count++;
  out.textContent = "Count: " + count;
});

// You handle every DOM update
// yourself.
```

Declarative (Vue.js)

```
<template>
  <button @click="count++">
    Count: {{ count }}
  </button>
</template>

<script setup>
import { ref } from 'vue'
const count = ref(0)
</script>

// Vue updates the DOM for you.
```

Vue.js — What Is It?

An approachable, performant, versatile framework

Vue.js

A progressive JavaScript framework for building user interfaces.

- Created by Evan You (2014)
- Current stable: Vue 3.x
- Official site: vuejs.org
- MIT open-source license

Key Features

1. **Declarative Rendering**

   HTML-like templates bound to reactive state.

2. **Reactivity System**

   Data changes automatically update the DOM.

3. **Component Based**

   Compose UIs from small, reusable parts.

4. **Progressive Adoption**

   Use a little or a lot — drop into any page or build a full SPA.

Vue vs React vs Angular

Where does Vue fit?

	Vue.js	React	Angular
Type	Framework	Library (UI)	Full framework
Template	HTML-based	JSX	HTML + directives
Learning curve	**Gentle**	Moderate	Steep
Size	~34 KB	~42 KB	~130 KB
Backed by	Community	Meta	Google

Vue blends React's component model with Angular's template familiarity — a great beginner-friendly choice.

Two Ways to Use Vue

Start simple, scale up when you need to

1. CDN (No Build Step)

Drop a <script> tag into any HTML page. Perfect for learning and small enhancements.

```
<script src="https://unpkg.com/vue@3">
</script>
<div id="app">{{ msg }}</div>
<script>
  const { createApp, ref } = Vue
  createApp({
    setup() {
      return { msg: 'Hello Vue!' }
    }
  }).mount('#app')
</script>
```

2. Build Tools (Vite / CLI)

Full project with Single File Components, routing, bundling. Covered in Chapter 7.

```
# Create a new Vue project
npm create vue@latest

# Install dependencies
cd my-app
npm install

# Start dev server
npm run dev

# -> Chapter 7
```

In Chapter 6 we use the CDN version so we can focus on Vue concepts, not tooling.

The Vue Application Instance

Every Vue app starts with createApp()

createApp(options)

creates a new Vue application instance.

.mount('#selector')

attaches it to an HTML element.

setup()

entry point of the Composition API — return reactive state & functions.

```
<!DOCTYPE html>
<html>
<body>
  <div id="app">
    <h1>{{ title }}</h1>
    <p>Visitors: {{ visitors }}</p>
  </div>

  <script src="https://unpkg.com/vue@3"></script>
  <script>
    const { createApp, ref } = Vue

    createApp({
      setup() {
        const title = ref('Welcome to Vue!')
        const visitors = ref(42)
        return { title, visitors }
      }
    }).mount('#app')
  </script>
</body>
</html>
```

Templating & Interpolation

Mustache syntax: {{ expression }}

Interpolation embeds a JavaScript expression inside HTML. The value updates automatically when the data changes.

```
<div id="app">
  <!-- simple value -->
  <p>Hello, {{ name }}!</p>

  <!-- JS expression -->
  <p>Total: {{ price * quantity }}</p>

  <!-- method call -->
  <p>{{ name.toUpperCase() }}</p>

  <!-- ternary -->
  <p>{{ age >= 18 ? 'Adult' : 'Minor' }}</p>
</div>
```

Rules of Interpolation

- One expression per {{ }} — no statements
- No if/for inside (use directives instead)
- Works only in TEXT, not attributes

  (use v-bind for attributes)
- Output is HTML-escaped by default

Reactive State — ref() and reactive()

How Vue knows when to update the DOM

ref() — for primitives

```
import { ref } from 'vue'

const count = ref(0)

// Access in JavaScript:
console.log(count.value) // 0
count.value++              // changes DOM!

// In template, no .value:
// <p>{{ count }}</p>
```

reactive() — for objects

```
import { reactive } from 'vue'

const user = reactive({
  name: 'Ali',
  age: 21
})

// No .value — access directly
user.age = 22   // triggers update

// In template:
// {{ user.name }} — {{ user.age }}
```

Rule of thumb: use ref() by default. It works for everything.

Core Directives Overview

Special v- attributes that bring HTML to life

v-bind

`:src="image"`

Bind an HTML attribute to a reactive value

v-on

`@click="go()"`

Listen to DOM events

v-model

`v-model="name"`

Two-way binding for form inputs

v-if

`v-if="isOpen"`

Conditionally render an element

v-for

`v-for="t in todos"`

Render a list from an array

v-show

`v-show="visible"`

Toggle CSS visibility (always rendered)

v-bind — Binding Attributes

Shorthand: :attribute="expression"

Use v-bind to dynamically set HTML attributes based on reactive state. Any change to the data updates the attribute automatically.

```
<!-- full form -->
<img v-bind:src="imageUrl" v-bind:alt="caption">

<!-- shorthand (most common) -->
<img :src="imageUrl" :alt="caption">

<!-- class binding -->
<p :class="{ active: isActive }">Hello</p>

<!-- style binding -->
<div :style="{ color: textColor, fontSize: size + 'px' }">...</div>

<!-- disable a button -->
<button :disabled="!canSubmit">Save</button>
```

v-on — Event Handling

Shorthand: @event="handler"

```
<!-- inline expression -->
<button v-on:click="count++">+1</button>

<!-- shorthand -->
<button @click="count++">+1</button>

<!-- call a method -->
<button @click="addItem">Add</button>

<!-- pass arguments -->
<button @click="remove(item.id)">Delete</button>

<!-- with modifiers -->
<form @submit.prevent="save">...</form>
<input @keyup.enter="search">
```

Useful modifiers

.prevent
event.preventDefault()

.stop
event.stopPropagation()

.once
fire only one time

.enter / .esc
key modifiers

v-model — Two-Way Binding

The data-entry superpower

v-model keeps a form input and a reactive value in sync: typing updates the data, and updating the data updates the input.

```
<input v-model="username" placeholder="Your name">
<p>Hello, {{ username }}!</p>

<!-- textarea, checkbox, radio, select -->
<textarea v-model="message"></textarea>
<input type="checkbox" v-model="agree">

<input type="radio" value="L"  v-model="size">
<input type="radio" value="XL" v-model="size">

<select v-model="country">
  <option>Malaysia</option>
  <option>Singapore</option>
</select>

<!-- modifiers: .lazy .number .trim -->
<input v-model.trim="email">
```

v-if, v-else-if, v-else, v-show

Conditional rendering

```
<!-- v-if / v-else-if / v-else -->
<p v-if="score >= 80">Excellent</p>
<p v-else-if="score >= 50">Good</p>
<p v-else>Keep trying</p>

<!-- v-show — always rendered, just toggles CSS display
-->
<div v-show="isMenuOpen">Menu</div>

<!-- group with <template> (no extra DOM) -->
<template v-if="isLoggedIn">
  <h2>Welcome back!</h2>
  <button @click="logout">Logout</button>
</template>
```

v-if vs v-show

v-if

adds/removes the DOM element

use for rare toggles

v-show

toggles display: none

use for frequent toggles

v-for — Rendering Lists

Loop through arrays and objects

```
<!-- Simple list -->
<ul>
  <li v-for="fruit in fruits" :key="fruit">
    {{ fruit }}
  </li>
</ul>

<!-- With index -->
<li v-for="(fruit, i) in fruits" :key="i">
  {{ i + 1 }}. {{ fruit }}
</li>

<!-- Array of objects -->
<tr v-for="s in students" :key="s.id">
  <td>{{ s.name }}</td>
  <td>{{ s.grade }}</td>
</tr>
```

:key is important!

Vue uses :key to track which item is which when the list changes.

Always provide a unique :key (usually an id).

Never use index as key when the list can reorder or items can be deleted.

Methods, Computed & Watchers

Functions, derived values, and side effects

Methods

Regular functions called on demand (events, template).

```
function addTodo() {
  todos.value.push(newTodo.value)
  newTodo.value = ''
}

<button
@click="addTodo">Add</button>
```

Computed

Derived value — cached and re-runs only when deps change.

```
import { computed } from 'vue'

const doneCount = computed(() =>
  todos.value.filter(t =>
t.done).length
)

<p>{{ doneCount }} done</p>
```

Watchers

Run side effects when a reactive value changes.

```
import { watch } from 'vue'

watch(username, (newVal, oldVal) =>
{
  console.log(`changed: ${oldVal} ->
${newVal}`)
})
```

Putting It All Together

A complete mini todo app in one file

TEMPLATE

```
<div id="app">
  <h2>Todos ({{ todos.length }})</h2>

  <input v-model="newTodo"
         @keyup.enter="add"
         placeholder="What to do?">
  <button @click="add">Add</button>

  <ul>
    <li v-for="(t,i) in todos" :key="i">
      <input type="checkbox" v-model="t.done">
      <span :class="{done:t.done}">{{ t.text
}}</span>
      <button @click="todos.splice(i,1)">x</button>
    </li>
  </ul>

  <p>{{ doneCount }} / {{ todos.length }}</p>
</div>
```

SCRIPT

```
<script src="https://unpkg.com/vue@3">
</script>
<script>
const { createApp, ref, computed } = Vue

createApp({
  setup() {
    const newTodo = ref('')
    const todos   = ref([])

    const doneCount = computed(() =>
      todos.value.filter(t => t.done).length
    )

    function add() {
      const t = newTodo.value.trim()
      if (!t) return
      todos.value.push({ text: t, done: false })
      newTodo.value = ''
    }
    return { newTodo, todos, doneCount, add }
  }
}).mount('#app')</script>
```

LAB Brief — Interactive Profile Card

Apply everything from today's lecture

You will build

- An HTML page using Vue 3 from CDN
- Editable profile (name, bio, skills)
- Live preview as you type (v-model)
- Toggle dark mode with :class binding
- Add/remove skills with v-for
- Show skill count with computed

Steps

1. **Create index.html with Vue CDN**
2. **Write the template**
3. **Add createApp({ setup() { ... } })**
4. **Declare reactive state with ref()**
5. **Bind inputs with v-model**
6. **Render skills with v-for**
7. **Add computed for skill count**
8. **Toggle theme class with @click**

Full guide in lab hand-out.

Summary — Key Takeaways

What to remember from Chapter 6

1. **Frameworks solve DOM complexity**

   Declarative UI > manual DOM manipulation.

2. **Vue is progressive & approachable**

   Start with a CDN, scale to full build tooling.

3. **createApp + mount + setup()**

   Every Vue app follows this shape.

4. **ref() makes values reactive**

   Changes to .value re-render the template.

5. **Directives drive interactivity**

   v-bind, v-on, v-model, v-if, v-for are your toolkit.

References & Next Steps

Official docs: https://vuejs.org/guide/introduction.html

Tutorial: https://vuejs.org/tutorial/

Style guide: https://vuejs.org/style-guide/

Vue Mastery (free courses): https://vuemastery.com

Next — Chapter 7

Building Full Vue Apps: CLI, Router, State Management

Terima kasih — Thank you!